

Challenge: Fleet Commander 2026

Welcome, Commanders. You manage **50 trucks**. Your mission is to process a week of hourly logs to find out who is efficient and who is stealing from the company.

Time Limit: 60 Minutes **Constraint:** Vanilla Python only (`dicts`, `lists`, `sets`, `zip`, etc.)

The Data Setup

Run this script first to generate your `fleet_data`. It creates a list of 50 dictionaries, each containing 168 hours (1 week) of logs.

```
Python
import random

def generate_data():
    fleet = []
    for t_id in range(100, 150):
        # Planned vs Actual
        truck = {
            "id": f"TRUCK-{t_id}",
            "plan": {"dist_km": 2500, "eta_hrs": 120, "consumption_rate": 16},
            "logs": [] # Hourly data
        }
        fuel = 400
        for h in range(168):
            dist_moved = random.uniform(15, 25)
            # Simulate Fuel Theft (Anomaly)
            fuel_loss = (dist_moved / 100) * truck["plan"]["consumption_rate"]
            if t_id == 105 and h == 20: fuel -= 50 # Theft event
            fuel -= fuel_loss

            truck["logs"].append({
                "hour": h,
                "gps_deviation": random.uniform(0, 15) if t_id != 110 else
random.uniform(0, 60),
                "fuel_level": fuel,
                "dist_covered": dist_moved,
                "road_block": random.choice([True, False]) if random.random() >
0.95 else False
            })
```

```
        fleet.append(truck)
    return fleet

fleet_data = generate_data()
```

Part 1: The Accountant (Reporting)

Goal: Create a summary dictionary of the fleet's performance.

Requirements:

1. Calculate **Total Fuel Consumed** and **Total Distance** per truck.
2. Calculate **Efficiency Score**: (Actual Consumption / Planned Consumption).
3. **Fleet Overview**: A final dictionary containing the average efficiency of all 50 trucks.

Target Output Format:

```
Python
report = {
    "TRUCK-100": {"total_dist": 3400, "efficiency": 0.95},
    "FLEET_AVG_EFFICIENCY": 0.98
}
```

Part 2: The Detective (Anomaly Detection)

Goal: Identify suspicious behavior. Use **configurable thresholds** so you can tune the sensitivity.

Requirements:

1. **Fuel Theft**: Detect if `fuel_level` drops by more than `X` liters in a single hour while `dist_covered` is near zero.
2. **Route Deviation**: Flag trucks where `gps_deviation` exceeds `Y` km for more than 3 consecutive hours.
3. **Output**: A dictionary of "Suspects" and the reason they were flagged.

Configurable Thresholds (Sane Defaults):

- `FUEL_DROP_LIMIT = 10` (Liters)
 - `GPS_DEV_LIMIT = 40` (Kilometers)
-

Success Criteria

- Your code must handle the nested structure of `fleet_data` using loops or comprehensions.
- The final output must be **one single dictionary** containing both the Report and the Anomalies. Beauty print dict. (JSON like)
- **No external libraries allowed.**

Suggestions for devs:

- Split the synthetic data generation into a separate py file - store that as JSON or pickle file.
- Create a py file that analyses the JSON or pickle truck fleet data, and outputs a beautiful JSON like structure with Report and Anomalies.